MASTER PROJECT PROPOSAL

Pre-Publication Ambiguity Detection in Internet-Drafts Using Revision-Aware Machine Learning and Large Language Models

Team Members	Mostafa Elofy Mohamed Ghanem Mohamed Abdelkhalek Osama Elkhuribi	**Student IDs**	20596321 20596317 20596298 20596292
Program	M.S. in Artificial Intelligence and Data Science	**Supervisor**	**Prof. Yuanzhu Chen**
Institution	School of Computing, Queen's University	**Co-Advisor**	**Dr. Ismail Anwar**

1. Abstract

Internet protocol standards evolve through successive Internet-Draft revisions before publication as Requests for Comments (RFCs). Ambiguous or under-specified requirements that survive this process can cause divergent implementations, interoperability failures, and costly clarification. Prior work has shown that natural language processing and large language models (LLMs) can expose ambiguity in published RFCs and software requirements, but it remains unclear whether ambiguity can be detected reliably during the draft stage, whether revision history available at the time of assessment provides useful signals, and when LLM reasoning justifies its cost. This project proposes a revision-aware, evidence-grounded framework for clause-level pre-publication ambiguity detection. Historical draft versions will be aligned with final RFC clauses, verified errata, and manually adjudicated revision outcomes to create gold, silver, and uncertain labels. Heuristic rules, representative classical machine-learning models, a fine-tuned transformer, an LLM-only baseline, and a hybrid routing pipeline will be evaluated using document-family and temporal holdout splits. The hybrid system will use a calibrated high-recall selector to route risky or uncertain clauses to an LLM verifier that cites evidence, assigns an ambiguity subtype, explains the decision, and may abstain. Expected contributions are a reproducible benchmark, a leakage-resistant comparative study, and a practical prototype for early standards review.

2. Introduction / Background

The Internet Engineering Task Force (IETF) specifies many Internet protocols in natural language. Normative keywords such as MUST, SHOULD, and MAY communicate requirement strength [1], [2], yet the surrounding prose may still contain inconsistent conditions, undefined references, missing cases, conflicting constraints, or under-specified behavior. Internet-Drafts are provisional working documents that may be revised, replaced, or expire before publication [3]. Their archived versions therefore provide a natural record of how technical requirements mature, making the draft stage an appropriate point for shift-left quality assurance.

Protocol-specific research demonstrates that ambiguity is not merely stylistic. SAGE used specification analysis and author clarification to expose ambiguous or under-specified statements before protocol code generation [5]. RFCScope developed a taxonomy of logical-ambiguity subtypes and an LLM-centered pipeline for published RFCs [4]. Requirements-engineering research has additionally explored linguistic rules, supervised learning, contextual transformers, requirement-smell detection, and LLM prompting [6]-[13]. These approaches differ in recall, evidence quality, reproducibility, latency, cost, and their ability to generalize beyond the documents used during development.

This project focuses on historical Internet-Draft versions and uses only information available up to the evaluated version. Its specific angle is to test whether contemporaneous revision signals and evidence-grounded hybrid routing improve ambiguity detection while reducing unnecessary LLM calls and maintaining a manageable workload for standards authors and reviewers.

3. Problem Statement

Existing studies primarily evaluate ambiguity detection on published RFCs or general software requirements. They do not establish whether ambiguous requirements can be identified reliably during the Internet-Draft stage using only information available at that time. It is also unclear whether revision-history and cross-reference features improve generalization, or whether an LLM-only approach provides sufficient benefit to justify its cost and instability relative to supervised classifiers. This project therefore formulates pre-publication ambiguity detection as a clause-level, evidence-grounded classification task and compares heuristic, machine-learning, transformer, LLM-only, and hybrid approaches under leakage-resistant evaluation.

4. Research Objectives

- Construct a clause-level benchmark linking historical Internet-Draft versions to final RFC text, verified technical errata, cross-document references, and adjudicated revision outcomes, with gold, silver, and uncertain labels.
- Define a reproducible annotation guide that distinguishes ambiguity clarification from editorial change, new functionality, terminology updates, reference corrections, and unrelated technical redesign.

- Implement one representative approach from each core family: heuristic rules, classical machine learning, a fine-tuned transformer, an LLM-only detector, and a hybrid ML-LLM routing framework.
- Prevent temporal and document leakage by using only contemporaneous input features and by evaluating with document-family and temporal holdout splits.
- Evaluate detection effectiveness, calibration, evidence faithfulness, latency, API/compute cost, and the proportion of clauses routed to LLM or human review.
- Deliver a reproducible prototype and empirical guidance on when LLM verification adds measurable value beyond conventional classification.

4.1 Research Questions

- RQ1: How effectively can ambiguous requirements be detected in historical Internet-Drafts using only information available at the evaluated draft version?
- RQ2: Do contemporaneous revision-history and cross-reference features improve F2, PR-AUC, and calibration over text-only models?
- RQ3: How do heuristic, classical ML, fine-tuned transformer, LLM-only, and hybrid approaches compare in effectiveness, reproducibility, latency, cost, and review workload?
- RQ4: Can hybrid routing preserve or improve detection effectiveness while reducing LLM calls relative to an LLM-only approach?
- RQ5: How faithful and useful are the evidence spans, ambiguity subtypes, and explanations produced by the LLM verifier?

4.2 Research Hypotheses

- H1: Contemporaneous revision-history features will improve F2 and PR-AUC over equivalent text-only models.
- H2: The hybrid pipeline will outperform the strongest non-LLM baseline in F2 or remain non-inferior to the LLM-only baseline while using fewer LLM calls.
- H3: Evidence grounding with neighboring clauses, definitions, and resolved references will reduce unsupported explanations and hallucinated citations.
- H4: Random clause splitting will produce significantly more optimistic results than document-family and temporal splits.

5. Related Work / Literature Review

Normative RFC language is structured by BCP 14 terminology [1], [2], yet requirement strength alone does not remove ambiguity. RFCScope is the closest protocol-specific baseline. It builds a taxonomy from verified technical errata, constructs targeted cross-document context, and uses bug-type-aware LLM analysis and validation to examine published RFCs [4]. Its strong results establish the feasibility of LLM-assisted detection, while its acknowledged limitations—context construction errors, hallucination, proprietary-model drift, and focus on published RFCs—motivate a comparative and pre-publication study. SAGE similarly showed that protocol specification text can be analyzed to expose ambiguity or under-specification before generating code, but its workflow depends on author clarification and targets executable protocol components rather than scalable draft screening [5].

Requirements-engineering work provides complementary methods. Early research defined linguistic sources of ambiguity and emphasized that harmful ambiguity depends on divergent stakeholder interpretations [11], [12]. Yang et al. combined linguistic heuristics with machine learning for nocuous coordination and anaphoric ambiguity [8], while Ezzini et al. evaluated multiple NLP, machine-learning, and BERT-based solutions for anaphoric ambiguity [7]. More recent work detects requirement smells and proposes controlled-language recommendations [6]. Izhar et al. combine rules and machine learning for software-requirement ambiguity [9], illustrating the continued relevance of engineered features and supervised classifiers. Bashir et al. evaluate in-context LLM ambiguity detection and explanation on industrial datasets, reporting gains from demonstrations but also limitations in domain terminology [10]. A systematic mapping study confirms that NLP support for requirements engineering spans rule-based, statistical, and neural approaches, with persistent challenges in datasets, generalization, and industrial validation [13].

The literature therefore leaves three connected opportunities: pre-publication evaluation on Internet-Draft histories; a controlled comparison between ML and LLM approaches; and leakage-resistant testing across time and protocol families. The proposed work addresses these opportunities without assuming that every revision is evidence of ambiguity or that LLMs are universally superior.

6. Proposed Approach / Methodology

The project will follow a five-stage workflow (Figure 1). First, a corpus builder will collect versioned Internet-Drafts, their final RFCs where available, document metadata, references, and verified errata. Second, a revision-alignment module will segment documents into normative and contextual clauses, match related clauses across versions, and compute textual and structural changes. Third, multiple candidate detectors will produce ambiguity scores. Fourth, an evidence-grounded LLM verifier will analyze selected high-risk or uncertain clauses with local context and resolved cross-document references. Fifth, an evaluation module will compare models and generate error analyses, calibration plots, cost summaries, and expert-review samples.

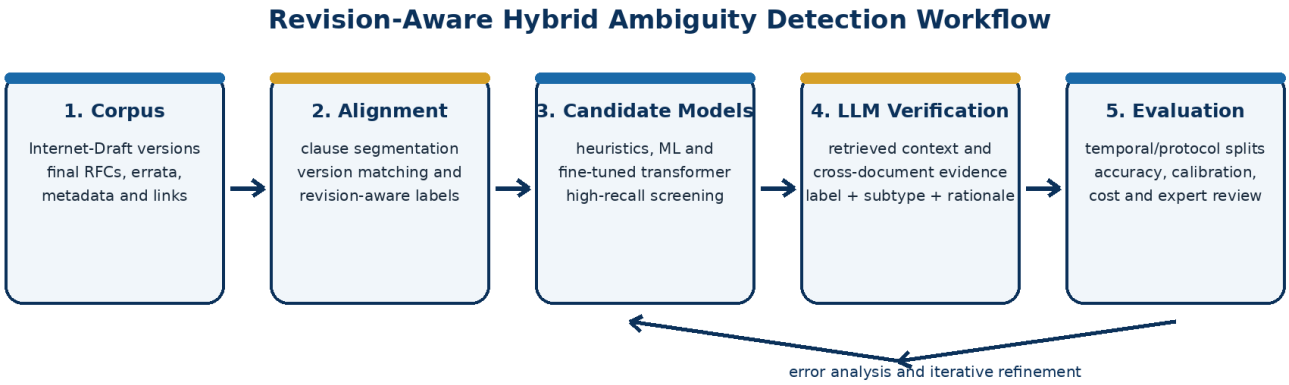


Figure 1. Proposed revision-aware hybrid ambiguity detection framework.

6.1 Candidate Detection Models

- Heuristic baseline: normative-keyword conflicts, vague quantifiers, undefined terms, unresolved references, complex conditionals, pronoun/anaphora patterns, exception density, and long dependency chains.
- Classical ML: logistic regression, support-vector machine, random forest, and gradient-boosted trees using linguistic, structural, cross-reference, and revision features.
- Fine-tuned transformer: a compact encoder such as DeBERTa or RoBERTa trained for ambiguous/unambiguous classification and, where data permits, ambiguity-subtype prediction.
- LLM-only baseline: zero/few-shot structured prompting with an explicit evidence span, subtype, confidence, and rationale.
- Hybrid model: a calibrated high-recall ML/transformer selector routes likely or uncertain candidates to the LLM verifier; clear low-risk clauses are resolved without an LLM call.

6.2 Revision-Aware Features and Evidence Grounding

Text features will include lexical uncertainty, syntactic complexity, semantic embeddings, modal patterns, negation, and conditional structure. Structural features will include section type, normative-keyword distribution, reference count, clause depth, table/figure dependencies, and document maturity. Revision features will include edit frequency, semantic distance between versions, changes in normative strength, clause stability, and proximity to publication. Retrieved evidence will be restricted to the target clause, neighboring context, referenced sections, and relevant definitions. The verifier must separate direct evidence from inference and may abstain when evidence is insufficient.

6.3 Data

Data will be drawn from public IETF and RFC Editor sources. All archived revisions of selected Internet-Draft families will be downloaded in machine-readable text/XML or HTML where available. Candidate families will be sampled across major protocol areas such as transport, routing, DNS, security, and application protocols. Final dataset size will depend on alignment quality and annotation capacity; the target is several thousand clause-version instances with a smaller, high-quality expert-reviewed test set.

Positive labels will be supported by verified technical errata, documented author clarifications, RFCScope-style ambiguity categories, or manual annotation showing that a later revision resolves multiple plausible interpretations. Negative labels will use stable, clear clauses from the same documents and must be reviewed to avoid treating absence of an erratum as proof of clarity. Each changed clause will also receive a revision-reason label: ambiguity clarification, editorial change, new functionality, terminology change, reference correction, or other technical redesign. Public documents will be used under their published access conditions; no personal or sensitive user data is required.

6.4 Implementation Plan

The prototype will be implemented in Python. Data collection and parsing will use requests, BeautifulSoup/lxml, and IETF XML tooling where appropriate. Text processing and alignment will use spaCy, sentence-transformers, and dynamic-programming or embedding-based matching. Classical models will use scikit-learn and gradient-boosting libraries; the transformer model will use PyTorch and Hugging Face Transformers. Experiment tracking will use MLflow or Weights & Biases, with a versioned dataset manifest and deterministic split files. LLM access will be abstracted behind a provider-independent interface so that at least one open-parameter model and one hosted model can be evaluated. A lightweight web or notebook interface will display the clause, retrieved evidence, score, subtype, rationale, and revision history for human review.

7. Evaluation Plan

Evaluation will avoid random clause splitting as the primary result because adjacent versions of the same draft are highly similar. The main protocols will be: (1) document-family split, keeping all versions of a draft family in one partition; (2) temporal split, training on earlier drafts and testing on later publication periods; and (3) protocol-family holdout, testing transfer to an unseen protocol area. Results will be reported for the complete test set and separately for ambiguity subtype, revision stage, and protocol family.

- Baselines: heuristic detector, majority/class-frequency baseline, logistic regression, SVM, random forest, gradient boosting, fine-tuned transformer, LLM-only prompting, and a reproducible RFCScope-inspired published-RFC baseline where feasible.

- Classification metrics: precision, recall, macro-F1, F2, PR-AUC, false-negative rate, and per-subtype F1. Because ambiguous clauses may be rare, accuracy will not be treated as the primary metric.
- Reliability and efficiency: expected calibration error, Brier score, risk-coverage curves for abstention, latency, token/compute cost, and percentage of clauses requiring LLM or human review.
- Explanation metrics: correctness of cited evidence, agreement between label and rationale, hallucinated-reference rate, and expert ratings of correctness, usefulness, and clarity on a stratified sample.
- Rigor: repeated runs with fixed split manifests, bootstrap confidence intervals, paired significance tests where appropriate, detailed error taxonomy, and ablation studies removing revision features, retrieval, subtype prompts, or the ML routing stage.

Success will not be defined only as a higher F1 score. The hybrid approach will be considered valuable if it offers a statistically supported improvement in recall or F2 over LLM-only and ML-only baselines while reducing LLM calls, maintaining evidence faithfulness, and producing a manageable review workload.

8. Expected Outcomes / Deliverables

- A documented, revision-aware Internet-Draft ambiguity dataset with clause alignments, labels, provenance, and fixed evaluation splits.
- Implementations of heuristic, classical ML, transformer, LLM-only, and hybrid ambiguity detectors.
- An evidence-grounded prototype for reviewing high-risk draft clauses and their revision history.
- A complete experimental report, ablation and error analysis, cost/latency study, and reproducibility package.
- A final master's thesis describing the literature, methodology, results, limitations, and recommendations for future standards-quality tooling.

9. Timeline

Phase	Task	Weeks
1	Literature review; refine the research gap, ambiguity taxonomy, project scope, annotation guidelines, and inclusion criteria	Week 1-3
2	Collect the Internet-Draft and RFC corpus; perform clause segmentation, version alignment, preprocessing, and pilot annotation	Week 4-7
3	Implement heuristic and classical machine-learning baselines; construct leakage-resistant temporal and protocol-level data splits; prepare the MVP	Week 8-9
4	Implement the transformer classifier, LLM-only approach, retrieval component, and hybrid ML–LLM ambiguity-detection pipeline	Week 10-12
5	Conduct evaluation, ablation studies, calibration analysis, expert review, statistical comparison, and detailed error analysis	Week 13–14
6	Finalize the software and code repository; complete thesis writing, revisions, artifact packaging, and final presentation	Week 15

10. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
Insufficient verified ambiguity labels	High	High	Use layered evidence: verified errata, author clarifications, RFCScope taxonomy, and dual-review manual annotation; publish uncertainty flags.
Revision changes are not caused by ambiguity	High	High	Annotate revision reason explicitly and exclude unverified edits from positive labels.
Clause alignment errors across versions	Medium	High	Combine structural anchors, similarity matching, and manual validation on the test set; report alignment accuracy.
LLM cost, drift, or non-reproducibility	Medium	Medium	Use cached outputs, fixed prompts, temperature-controlled runs, open-parameter baselines, and versioned model metadata.
Class imbalance and optimistic leakage	High	High	Use family/time/protocol splits, PR-based metrics, class weighting, and negative sampling within the same documents.
Scope exceeds available time	Medium	High	Prioritize binary detection and comparative evaluation; treat automated revision recommendations as optional future work.

11. Resources Required

- Compute: one modern GPU for transformer fine-tuning and local LLM experiments; limited cloud/API credits for hosted LLM comparisons.
- Software: Python and open-source NLP/ML libraries; Git-based version control; experiment tracking and artifact storage.
- Data: public IETF Internet-Draft archive, IETF Datatracker metadata, RFC text, and RFC Editor errata; no private access agreement is expected.
- Human input: periodic advisor review, a pilot annotation workshop, and at least two knowledgeable reviewers for the high-quality test subset and explanation assessment.

- Recommended cadence: weekly team progress review and advisor synchronization every two weeks, with milestone reviews after dataset completion and baseline evaluation.

12. References

[1] S. Bradner, “Key words for use in RFCs to indicate requirement levels,” IETF, RFC 2119, Mar. 1997, doi: 10.17487/RFC2119.

[2] B. Leiba, “Ambiguity of uppercase vs lowercase in RFC 2119 key words,” IETF, RFC 8174, May 2017, doi: 10.17487/RFC8174.

[3] IETF, “Guidelines to Authors of Internet-Drafts,” 2026. [Online]. Available: IETF Authors documentation.

[4] M. Pawagi, L. Shao, H. Lee, Y. Sun, and W. Wang, “RFCScope: Detecting logical ambiguities in Internet protocol specifications,” in Proc. 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2025, pp. 1234-1246, doi: 10.1109/ASE63991.2025.00106.

[5] J. Yen, T. Lévai, Q. Ye, X. Ren, R. Govindan, and B. Raghavan, “Semi-automated protocol disambiguation and code generation,” in Proc. ACM SIGCOMM, 2021, pp. 272-286.

[6] A. Veizaga, S. Y. Shin, and L. C. Briand, “Automated smell detection and recommendation in natural language requirements,” IEEE Trans. Software Engineering, vol. 50, no. 4, pp. 695-720, 2024, doi: 10.1109/TSE.2024.3361033.

[7] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, “Automated handling of anaphoric ambiguity in requirements: A multi-solution study,” in Proc. 44th IEEE/ACM Int. Conf. Software Engineering (ICSE), 2022, pp. 187-199.

[8] H. Yang, A. De Roeck, V. Gervasi, A. Willis, and B. Nuseibeh, “Analysing anaphoric ambiguity in natural language requirements,” Requirements Engineering, vol. 16, pp. 163-189, 2011.

[9] H. Yang, A. Willis, A. De Roeck, and B. Nuseibeh, “A methodology for automatic identification of nocuous ambiguity,” in Proc. 23rd Int. Conf. Computational Linguistics (COLING), 2010, pp. 1218-1226.

[10] R. Izhar, S. N. Bhatti, and S. A. Alharthi, “Bridging precision and complexity: A novel machine learning approach for ambiguity detection in software requirements,” IEEE Access, vol. 13, pp. 12014-12031, 2025, doi: 10.1109/ACCESS.2025.3529943.

[11] S. Bashir, A. Ferrari, M. A. Khan, P. E. Strandberg, Z. Haider, M. Saadatmand, and M. Bohlin, “Requirements ambiguity detection and explanation with LLMs: An industrial study,” in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2025, pp. 620-631.

[12] D. M. Berry, E. Kamsties, and M. M. Krieger, “From contract drafting to software specification: Linguistic sources of ambiguity,” School of Computer Science, University of Waterloo, Tech. Rep., 2003.

[13] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, “Natural language processing for requirements engineering: A systematic mapping study,” ACM Computing Surveys, vol. 54, no. 3, pp. 1-41, 2021.